Guided Differential Evolution through history sliding population

No Author Given

[1] Depto. de Ciencias Computacionales, Universidad de Guadalajara, CUCEI, Guadalajara, Jalisco, Mexico, C.P.44430
arturo.valdivia@academicos.udg.mx, ignacio.pena0184@alumnos.udg.mx, {itzel.aranguren@academicos.udg.mx,
[2] Depto. de Ingeniería Electro-Fotónica, Universidad de Guadalajara, CUCEI, Guadalajara, Jalisco, Mexico, C.P.44430
diego.oliva@cucei.udg.mx, angel.casas5699@alumnos.udg.mx

Abstract. Differential evolution remains among the most effective evolutionary algorithms for global optimization, yet its performance depends on balancing exploration and exploitation. This paper proposes Guided Differential Evolution (GDE), an enhanced framework that addresses the exploration-exploitation trade-off through two main contributions: 1) a bimodal mutation scheme that alternates deterministically between exploration and exploitation phases, and 2) a lightweight guidance mechanism that uses a sliding-window history of the population to steer candidate solutions toward a central trend. Unlike standard DE, GDE incorporates historical trajectory information to stabilize convergence without increasing computational complexity. Retaining the classical DE structure, GDE augments it with a guidance mechanism and deterministic phase scheduling. Experimental validation on the CEC'2017 benchmark suite confirms competitive performance across diverse function categories, demonstrating the effectiveness of the proposed phase alternation strategy.

Keywords: Differential Evolution · Guided Mutation · programmed exploitation and exploration· Metaheuristic algorithms

1 Introduction

Differential Evolution (DE) [13] stands as one of the most representative algorithms in evolutionary computation, owing to its simple structure, low computational overhead, and competitive performance on continuous optimization problems [9]. Nevertheless, its effectiveness is heavily contingent upon the appropriate configuration of mutation strategies and control parameters, as these govern the search dynamics throughout the evolutionary process. Consequently, numerous variants have been proposed targeting key components such as the scale factor, crossover rate, mutation operators, and selection criteria [11].

Among these components, the mutation stage occupies a central role, as it confers upon DE its distinctive "differential" character by constructing candidate solutions through vector combinations drawn from the population. The

selected mutation strategy directly shapes both the search direction and convergence behavior. It is well established, however, that no single strategy universally dominates across all problem types or evolutionary stages [8].

In response to these limitations, this article presents a Guided differential evolution (GDE) framework that introduces a bimodal strategy guided by historical population dynamics while preserving the essential structure of DE. By incorporating a sliding historical window as temporal memory, the GDE framework strategically alternates between an exploration-oriented mutation and a guided phase designed for consistent solution refinement.

The main contributions of this work can be summarized in the following points:

- A bimodal mutation scheme is proposed within Differential Evolution, establishing a clear alternation between an exploration-oriented stage and an exploitation-focused stage.
- A mechanism based on scaling factor variation is introduced, allowing transitions between these two modes without altering the classical structure operators of Differential Evolution.
- A lightweight temporary memory component is incorporated using a sliding historical window, which benefits from accumulated population information to guide the mutation process.

The rest of the paper is structured as follows: Section 2 describes related works and DE algorithm. Section 3 details the proposal. Section 4 presents the results for the CEC'2017 benchmark. Last, Section 5 resumes the main findings and future directions.

2 Background and Related Work

2.1 Differential Evolution

Differential Evolution (DE), proposed by Rainer Storn and Kenneth Price in 1997, has become one of the most widely used evolutionary algorithms for optimization problems. DE operates in three fundamental stages: mutation, crossover, and selection, which regulate the balance between exploration and exploitation throughout the evolutionary process. The algorithm works on a set of parameter vectors, where each vector represents a possible solution to the problem.

1. In this stage, each vector in the population, called the target vector, gives rise to a new mutant vector. To construct it, three different individuals are randomly selected from the current population. From these, the vector difference between any two of the selected vectors is calculated; this difference is multiplied by a scaling factor F and then added to the third vector. This mechanism allows for the generation of new candidate solutions that explore the search space. The most commonly used variant at this stage is

the strategy known as DE/rand/1. The parameter F acts as a scaling factor and controls the magnitude of the perturbation introduced in the mutation process.

2. To combine the genetic information of the mutant vector with the target vector, the crossover stage is performed, generating a trial vector that inherits its characteristics from both. In this phase, a new vector is constructed by combining components of the mutant vector with those of the original target vector. This process allows for the mixing of genetic information and the generation of potentially more competitive solutions. Crossover is regulated by a parameter called the crossover rate (CR), whose value is usually determined empirically based on the characteristics of the problem. This parameter defines the probability that each component of the new vector comes from the mutant vector rather than the target vector. The result of this operation is the test vector $u^d_{i,G+1}$. Additionally, a random index d_{rand} is used to ensure that at least one of the components of the test vector is inherited from the mutant vector, thus preventing the solution from remaining unchanged.
3. Finally, the selection stage determines which solution continues into the next generation. In this step, the trial vector obtained after the crossover is compared to the original target vector in terms of its fitness value. If the new vector performs better then it replaces the target vector in the next iteration. Otherwise, if its fitness is less competitive, the original solution is retained. This selection mechanism ensures that the population evolves progressively toward higher-quality regions in the search space, while maintaining the stability of the evolutionary process.

2.2 Recent Advances in DE Mutation

With the aim of diversifying the set of mutation operators in differential evolution (DE), recent research has adopted multi-strategy approaches that integrate several mutation variants and assign them adaptively throughout the evolutionary process. In [14], an adaptive mutation operator is proposed based on fitness landscape analysis, where a random forest model is used to predict, during evolution, which strategy is most suitable depending on the behavior of the problem. The results indicate that the method achieves competitive performance against different DE variants; however, its implementation involves a complex structure and requires prior offline training with specific functions. In [7], an improved elite-guided scheme is introduced, which develops two mutation strategies based on elite population vectors to accelerate convergence without sacrificing diversity. This mechanism provides a more direct path to the optimum without eliminating the stochastic component.

3 Proposed approach

The algorithm introduced in this work is governed by a structured programming scheme, formally defined as $V_{mode} = \{(M_1, I_1), (M_2, I_2), \ldots\}$, which partitions

the evolutionary process into two operationally distinct phases. Within this formulation, each element M designates the mutation strategy applied to the entire population at a given stage, whereas I specifies the number of consecutive generations for which that particular phase remains in effect.

This design imposes an explicit temporal structure upon the evolutionary dynamics, ensuring that the transitions between operational phases occur in a deterministic and reproducible manner. A fundamental constraint of this scheduling mechanism is that the cumulative sum of all interval durations I must equal the maximum number of iterations It_{max}, as formally stated in Equation 1. This condition guarantees complete coverage of the evolutionary horizon prescribed for the algorithm.

$$It_{max} = \sum_{k=1}^{n} I_k \tag{1}$$

where n denotes the total number of tuples stored in V_{mode}.

3.1 Population Initialization and History Setup

The initialization phase constructs a population of NP individuals distributed across the feasible search (Algorithm 1, Line 1), defined by lower and upper bounds $[L, U]$, over a D-dimensional problem domain. Additionally, an empty history buffer H is instantiated to record the state of the entire population (Algorithm 1, Line 2) at each discrete time step t throughout the evolutionary process.

3.2 Exploration Mode: Mutation Type 1

During exploration, the algorithm follows the canonical Differential Evolution framework, in which each target vector x_i is processed through the classical DE/rand/1 scheme (Algorithm 1, Lines 6–11) with binomial crossover and greedy selection [13]:

1. **Stochastic Mutation:** For each target vector x_i, three mutually distinct individuals are randomly selected from the current population to construct a mutant vector. A relatively high scaling factor $F = 0.83$ (Algorithm 1, Line 8), as specified in Table 3, is employed to promote broad exploration of the search space.
2. **Crossover and Selection:** Standard binomial crossover is applied in conjunction with greedy selection (Algorithm 1, Lines 9–10) to guide the population toward regions of lower objective function value, while preserving sufficient genetic diversity to prevent premature convergence.

3.3 Guided Exploitation Mode: Mutation Type 2

Upon activation by the scheduling vector V_{mode}, the search strategy transitions from stochastic perturbation-based exploration to a collective trend-guided exploitation mechanism (Algorithm 1, Lines 6–11). The term $\bar{G}_e$ represents the centroid of the population computed over the most recent ω generations (Algorithm 1, Line 14) and is formally expressed as:

$$\bar{G}_e = \frac{1}{\omega \cdot NP} \sum_{g=t-\omega}^{t} \sum_{i=1}^{NP} x_{i,g} \tag{2}$$

where t is the index of the current generation, g serves as a counter iterating over the generations retained in the history buffer, and ω denotes the temporal window size, set to $\omega = 6$ throughout this study.

Following the computation of $\bar{G}_e$, the mutant vector v_i is constructed according to the formulation presented in Equation 3:

$$v_i = (1 - \lambda) \cdot [x_{r_1} + F_{low}\,(x_{r_2} - x_{r_3}) + F_{low}\,(x_{r_4} - x_{r_5})] + \lambda \cdot \bar{G}_e \tag{3}$$

In Equation 3, $r_1, r_2, r_3, r_4, r_5 \in \{1, \ldots, NP\}$ are mutually exclusive random indices chosen from the current population, which are also distinct from the target vector index i. The term λ denotes the guidance intensity parameter, assigned a value of $\lambda = 0.21$. This formulation attenuates high-frequency stochastic perturbations inherent to random mutation, effectively replacing erratic displacement patterns with a regularized descent trajectory oriented toward the global optimum. It is worth noting that the scheduling vector V_{mode} repeats its defined pattern five times in order to align with the total generation budget stipulated in the CEC'2017 benchmark suite for the 50-dimensional configuration, corresponding to $10,000$ function evaluations. The complete phase scheduling pattern is detailed in Table 1, and the full algorithmic procedure is presented in Algorithm 1.

The specific parameter settings for GDE, detailed in Table 3, were selected empirically based on preliminary tuning experiments. These tests were conducted to ensure that the bimodal scheduling provides an effective transition between stochastic exploration and guided refinement. Similarly, the iteration counts for each phase in V_{mode} (Table 1) were determined through empirical observation to maximize the benefits of the historical population dynamics across the CEC'2017 benchmark suite.

Table 1: V_{mode} of GDE

Index	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
Mutation	R1	GM	R1	GM	R1	GM	R1	GM	R1	GM	R1	GM	R1	GM	R1	GM	R1	GM	R1	GM
Iterations	120	80	120	80	120	80	120	80	120	80	80	120	80	120	80	120	80	120	80	120

Algorithm 1 Guided Differential Evolution (GDE)

Require: NP, $It_{\max}$, V_{mode} = list of (phase, n_iters) pairs with $\sum n_{iters} = It_{\max}$
Require: $F_{\text{high}} \leftarrow 0.83$, $F_{\text{low}} \leftarrow 0.31$, $CR \leftarrow 0.21$, $\lambda \leftarrow 0.21$, $w \leftarrow 6$
Initialize $P^{(0)}$, evaluate $f_i^{(0)}$, $B^{(0)} \leftarrow \min f_i^{(0)}$
`best_hist` $\leftarrow [B^{(0)}]$, `history` $\leftarrow [P^{(0)}]$
$g \leftarrow 0$
for each (phase, n) $\in V_{mode}$ **do**
  for $k = 1$ to n **do**
    $g \leftarrow g + 1$
    if phase = `"Exploration"` **then**
      for $i = 1$ to NP **do**
        $v_i \leftarrow$ DE/rand/1 mutation$(P^{(g-1)}, F_{\text{high}})$
        $u_i \leftarrow$ binomial crossover$(P_i^{(g-1)}, v_i, CR)$
        Evaluate $f(u_i)$; greedy replace if better
        if $f(u_i) \leq f(P_i^{(g-1)})$ **then**
          $P_i^{(g)} \leftarrow u_i$ ▷ Update individual in the new population
        else
          $P_i^{(g)} \leftarrow P_i^{(g-1)}$
        end if
      end for
    else ▷ Guided Exploitation
      $\bar{G_e} \leftarrow$ average of last $\min(w, |$`history`$|)$ populations in `history`
      for $i = 1$ to NP **do**
        Choose distinct $r1, r2, r3, r4, r5$
        $v_i \leftarrow (1 - \lambda) \cdot [x_{r1} + F_{\text{low}}(x_{r2} - x_{r3}) + F_{\text{low}}(x_{r4} - x_{r5})] + \lambda \cdot \bar{G_e}$
        $u_i \leftarrow$ binomial crossover$(P_i^{(g-1)}, v_i, CR)$
        Evaluate $f(u_i)$; greedy replace if better
        if $f(u_i) \leq f(P_i^{(g-1)})$ **then**
          $P_i^{(g)} \leftarrow u_i$ ▷ Update individual in the new population
        else
          $P_i^{(g)} \leftarrow P_i^{(g-1)}$
        end if
      end for
    end if
    $B^{(g)} \leftarrow \min_i f(P_i^{(g)})$ ▷ Explicitly calculate best fitness of current gen
    Append $B^{(g)}$ to `best_hist`
    Append $P^{(g)}$ to `history`
  end for
end for
return `best_hist`

4 Experimental Results

In the experiments conducted, the CEC'2017 benchmark set was used, widely recognized in the literature for evaluating the performance of evolutionary algorithms in numerical optimization problems. A detailed description of these

functions can be found in Table 2. The algorithms considered in the comparison, as well as the parameters used in each case, are presented in Table 3. In order to perform a comprehensive evaluation, all experiments were carried out in a 50-dimensional space, running 30 independent repetitions per function, with a budget of 10,000 iterations and a population of 50 individuals, following the recommendations established in [1]. The implementation of GDE was developed in Python 3.9, and all tests were run under the same computational conditions: a machine with four 3.7 GHz AMD® Ryzen 9 processors, 32 GB of RAM, and the Ubuntu 24.04.4 LTS operating system. The link to a version of GDE is available at https://github.com/espartan0007/GDE .

Table 2: Objective functions defined in the CEC'2017 benchmark suite.

Type	No	Benchmark function	$f(x)$
Unimodal	1	Shifted and Rotated Bent Cigar	100
	2	Shifted and Rotated Sum of Different Power	200
	3	Shifted and Rotated Zakharov	300
Multimodal	4	Shifted and Rotated Rosenbrock's	400
	5	Shifted and Rotated Rastrigin's	500
	6	Shifted and Rotated Expanded Scaffer's F6	600
	7	Shifted and Rotated Lunacek Bi_ Rastrigin	700
	8	Shifted and Rotated Non-Continuous Rastrigin's	800
	9	Shifted and Rotated Levy	900
	10	Shifted and Rotated Schwefel's	1000
Hybrid	11	Hybrid Function 1 ($N = 3$)	1100
	12	Hybrid Function 2 ($N = 3$)	1200
	13	Hybrid Function 3 ($N = 3$)	1300
	14	Hybrid Function 4 ($N = 4$)	1400
	15	Hybrid Function 5 ($N = 4$)	1500
	16	Hybrid Function 6 ($N = 4$)	1600
	17	Hybrid Function 6 ($N = 5$)	1700
	18	Hybrid Function 6 ($N = 5$)	1800
	19	Hybrid Function 6 ($N = 5$)	1900
	20	Hybrid Function 6 ($N = 6$)	2000
Composition	21	Composition Function 1 ($N = 3$)	2100
	22	Composition Function 2 ($N = 3$)	2200
	23	Composition Function 3 ($N = 4$)	2300
	24	Composition Function 4 ($N = 4$)	2400
	25	Composition Function 5 ($N = 5$)	2500
	26	Composition Function 6 ($N = 5$)	2600
	27	Composition Function 7 ($N = 6$)	2700
	28	Composition Function 8 ($N = 6$)	2800
	29	Composition Function 9 ($N = 3$)	2900
	30	Composition Function 10 ($N = 3$)	3000

Note: The column $f(x)$ represents the known global optimum value (fitness bias) for each function according to the CEC'2017 specifications. This value is used as the baseline for performance comparison.

The performance of each algorithm was evaluated using three statistical indicators: the Best fitness value, Average Best fitness and the Standard Deviation, computed from the results obtained over 30 independent runs per function. The Average Best reflects the overall solution quality achieved by each method, while the Standard Deviation provides insight into the robustness and stability of the algorithm across repeated executions. To facilitate comparison, the best numer-

Table 3: Parameter settings of the compared algorithms

Algorithm	Parameter	Value
GDE	Scaling factor Mode 1 F	0.83
	Scaling factor Mode 2 F_l	0.31
	Crossover rate CR	0.21
	Historic Window ω	6
	Guidance λ	0.21
	Vector of the scheduling V_{mode}	See table 1
PSO [6]	Acceleration coefficients c_1, c_1	1.2
	Inertia weight ω	0.9
DE [13]	Scaling factor F	0.8
	Crossover rate C_r	0.2
JADE [15]	Global memory H	10
	Archive factor r_{arc}	$1.4 \cdot N$
	Pbest rate p	0.11
SaDE [10]	Learning Generations LS_{Gens}	25
	Normal mean (CR) μ	0.5
	Normal std (CR) σ^2	0.1
	Normal mean (F) μ	0.5
	Normal std (F) σ^2	0.3

ical results for each function are highlighted in bold in Table 4 corresponding respectively to unimodal and multimodal, hybrid, and composition functions.

To analyze the best results obtained in Tables 4, 5 and 6, we highlighted the numerical values in bold to identify them quickly.

Table 4 presents the results obtained on the CEC'2017 unimodal (F01–F03) and multimodal (F04–F10) functions in 50 dimensions. In the unimodal problems, GDE achieves competitive performance in terms of Best Fitness, and Average, while maiwhereas the Standard Deviation provides insight into the algorithm's robustness and stability fine solutions in a stable and consistent manner. For the multimodal functions, GDE obtains the best results in several cases (such as F04, F06, F07, and F10) and remains competitive in the remaining ones. Although some functions are better solved by other methods, GDE generally shows balanced behavior and reduced variability across runs compared to classical DE and PSO. The convergence curves in Fig. 1 illustrate this behavior, where GDE demonstrates steady progress and stable convergence throughout the optimization process.

Table 4: Statistical results for CEC2017 unimodal and multimodal functions

Function	Metrics	DE	SaDE	PSO	JADE	GDE
F01	Best Fitness	163.48	106.68	4.54E+09	3.91E+08	**100.06**
	Average	7151.69	1.02E+04	2.45E+10	2.09E+09	**5428.99**
	Standard Deviation	7376.58	46000.00	1.14E+10	1.30E+09	**5762.23**
F02	Best Fitness	**200.00**	**200.00**	**200.00**	1.11E+04	**200.00**
	Average	**200.02**	241.69	9889.64	2.76E+04	**200.00**
	Standard Deviation	0.03	194.00	8418.56	8234.02	**4.22E-06**
F03	Best Fitness	**356.14**	369.46	908.23	635.93	**356.14**
	Average	428.18	437.46	4367.02	899.00	**410.64**
	Standard Deviation	40.43	37.53	2565.16	173.26	**37.48**
F04	Best Fitness	770.90	927.01	6812.46	2892.19	**466.66**
	Average	**806.11**	2660.11	2.62E+04	7142.66	1179.83
	Standard Deviation	**18.71**	1112.65	1.40E+04	2339.79	936.95

Continued on next page...

Table 4 – Continued from previous page

Function	Metrics	DE	SaDE	PSO	JADE	GDE
F05	Best Fitness	**500.00**	**500.00**	**500.00**	**500.00**	**500.00**
	Average	**500.00**	**500.00**	**500.00**	**500.00**	**500.00**
	Standard Deviation	3.77E-03	4.21E-05	1.90E-03	1.76E-05	**5.40E-06**
F06	Best Fitness	947.94	1485.18	5.17E+05	4.36E+04	**739.98**
	Average	**1057.78**	4378.76	1.48E+06	1.05E+05	2674.29
	Standard Deviation	**30.24**	2132.96	6.19E+05	5.04E+04	1537.24
F07	Best Fitness	994.29	840.04	1218.00	844.00	**788.00**
	Average	1449.48	**890.64**	1404.02	916.82	1226.05
	Standard Deviation	121.19	**29.58**	103.47	42.25	173.44
F08	Best Fitness	**800.00**	803.54	820.47	802.33	801.08
	Average	**800.07**	807.51	834.90	806.19	803.07
	Standard Deviation	**0.18**	2.52	8.84	1.94	1.00
F09	Best Fitness	1770.73	5356.09	**19.59**	3430.61	5357.71
	Average	4390.65	6813.56	**2281.35**	5641.56	6816.63
	Standard Deviation	1864.15	1092.34	**1570.82**	985.45	1012.31
F10	Best Fitness	1240.33	4073.49	1.36E+05	4.03E+04	**1222.81**
	Average	**1346.96**	6565.37	1.54E+07	7.89E+04	1965.27
	Standard Deviation	**44.90**	2029.89	2.18E+07	2.16E+04	443.20

Table 5 presents the statistical results for the CEC'2017 hybrid functions (F11–F20) in 50 dimensions. These problems are more challenging, as they combine different landscape characteristics within a single formulation, requiring both effective exploration and stable refinement. From the results, GDE achieves the best Best Fitness values in several cases (such as F13, F14, F16, F17, F19, and F20) and remains competitive in others. In particular, for functions like F16 and F17, GDE shows lower average values with relatively small standard deviations, indicating stable performance across runs. Although in some functions (e.g., F11 and F12) SaDE attains better results, GDE maintains balanced behavior and avoids the large variability observed in PSO and, in several cases, JADE. Overall, the results suggest that the guided phase alternation helps the algorithm handle the structural complexity of hybrid functions in a consistent manner.

Table 5: Statistical results for CEC2017 hybrid functions

Function	Metrics	DE	SaDE	PSO	JADE	GDE
F11	Best Fitness	2.43E+05	**12900.00**	3.03E+06	2.36E+06	13600.00
	Average	1.35E+06	**34200.00**	1.63E+09	1.93E+07	1.30E+06
	Standard Deviation	9.74E+05	**18900.00**	1.99E+09	1.27E+07	1.79E+06
F12	Best Fitness	3970.43	**2716.81**	7.91E+07	3.70E+05	4856.64
	Average	4.39E+04	**15900.00**	6.97E+09	2.48E+07	27900.00
	Standard Deviation	3.27E+04	**7257.52**	8.31E+09	3.47E+07	22200.00
F13	Best Fitness	2452.05	2.05E+04	4.24E+04	9.14E+04	**2031.22**
	Average	**4553.83**	1.10E+05	2.95E+06	3.52E+05	12000.00
	Standard Deviation	**2455.08**	1.11E+05	7.26E+06	1.91E+05	9115.92
F14	Best Fitness	4196.20	5940.68	1.98E+05	6.65E+04	**3037.59**
	Average	2.45E+04	1.63E+04	1.89E+09	7.31E+05	**15500.00**
	Standard Deviation	1.67E+04	**6448.78**	2.64E+09	1.80E+06	7839.72
F15	Best Fitness	**996.45**	1274.00	8.68E+04	6729.14	1088.61
	Average	**1630.53**	2975.80	4.61E+08	9.20E+04	2059.70
	Standard Deviation	2389.88	**1987.42**	8.06E+08	1.24E+05	2448.38
F16	Best Fitness	2040.26	1524.14	3480.16	1.20E+04	**1196.69**
	Average	3167.30	3262.90	8.72E+04	2.86E+04	**2049.13**
	Standard Deviation	790.67	1121.97	5.14E+04	1.17E+04	**760.97**

Continued on next page...

Table 5 – Continued from previous page

Function	Metrics	DE	SaDE	PSO	JADE	GDE
F17	Best Fitness	4614.03	8789.22	6.23E+04	1.01E+05	**3059.25**
	Average	1.55E+04	5.07E+04	4.22E+05	2.05E+05	**10900.00**
	Standard Deviation	**6362.75**	3.90E+04	3.41E+05	7.94E+04	9007.38
F18	Best Fitness	**1869.07**	1918.21	6.76E+04	4.80E+04	1949.95
	Average	**1901.56**	5392.19	1.10E+12	3.32E+07	5217.36
	Standard Deviation	**15.41**	2622.97	4.67E+12	1.31E+08	3043.61
F19	Best Fitness	1781.95	1850.18	1890.44	1789.23	**1767.34**
	Average	**1930.83**	2049.17	3775.19	2138.52	1998.31
	Standard Deviation	126.28	**114.89**	1193.35	155.32	149.91
F20	Best Fitness	2100.01	2100.00	8101.70	3068.44	**2100.00**
	Average	**2849.52**	4118.75	2.20E+04	6078.49	3132.06
	Standard Deviation	**622.11**	1219.13	1.11E+04	2239.53	1394.09

Table 6 presents the statistical results for the CEC'2017 composition functions (F21–F29) in 50 dimensions. These functions are particularly demanding, as they combine multiple landscape characteristics within expanded search domains, requiring both strong exploration and stable convergence. From the results, GDE achieves the best Best Fitness values in several cases (notably F22, F23, F24, F25, and F29) and remains competitive in others. In functions such as F28 and F29, GDE also attains the best or among the best Average and Standard Deviation values, indicating consistent behavior across independent runs. Although in some instances other algorithms obtain slightly better average or best-case results (e.g., DE in F21 and F27, JADE in F26), GDE generally maintains balanced performance without exhibiting the large variability observed in PSO and, in several cases, JADE. Overall, the results suggest that the guided phase alternation allows the algorithm to handle the structural complexity of composition functions in a stable and reliable manner.

Table 6: Statistical results for CEC2017 compose functions

Function	Metrics	DE	SaDE	PSO	JADE	GDE
F21	Best Fitness	2.14E+12	**2.24E+11**	-5.44E+12	2.07E+12	2.65E+11
	Average	1.58E+13	**1.51E+13**	7.15E+12	1.15E+13	1.25E+13
	Standard Deviation	6.84E+12	**9.61E+12**	9.42E+12	9.38E+12	1.13E+13
F22	Best Fitness	2400.02	2400.01	5140.21	4438.79	**2400.00**
	Average	4999.29	**4271.06**	3.03E+04	1.13E+04	4999.67
	Standard Deviation	4846.35	**2224.16**	1.20E+04	4550.79	4847.34
F23	Best Fitness	2500.01	2500.00	9990.67	3681.20	**2500.00**
	Average	2500.02	3032.11	2.13E+04	6525.57	**2500.00**
	Standard Deviation	**7.90E-03**	1131.62	5812.49	1674.74	0.01
F24	Best Fitness	3106.23	3155.07	3462.22	3306.20	**3103.39**
	Average	**3125.32**	3290.68	5722.06	3918.73	3175.55
	Standard Deviation	**9.17**	83.87	2143.85	276.34	67.21
F25	Best Fitness	3690.08	3789.44	5015.39	3987.36	**3685.86**
	Average	3998.47	3989.06	6460.13	4223.55	**3977.97**
	Standard Deviation	140.43	**103.45**	1122.64	142.37	171.65
F26	Best Fitness	3049.82	3043.90	3278.94	**3029.41**	3080.24
	Average	**3099.62**	3135.94	3561.68	3119.62	3160.94
	Standard Deviation	**41.90**	59.32	230.53	46.70	49.42
F27	Best Fitness	**2700.01**	2841.02	4000.59	3110.88	2832.36
	Average	**2710.51**	3067.56	4557.66	3209.28	2970.08
	Standard Deviation	56.63	88.94	443.21	**31.44**	68.32
F28	Best Fitness	**6216.35**	1.25E+04	5.78E+05	9.97E+05	6618.60
	Average	**8276.72**	6.28E+04	1.63E+09	2.48E+07	9580.16
	Standard Deviation	1540.36	7.02E+04	2.92E+09	3.34E+07	**1463.78**

Continued on next page...

Table 6 – Continued from previous page

Function	Metrics	DE	SaDE	PSO	JADE	GDE
F29	Best Fitness	1.10E+05	2.95E+04	1.45E+07	2.64E+04	**13700.00**
	Average	3.59E+05	5.97E+04	1.58E+11	2.48E+07	**31800.00**
	Standard Deviation	2.24E+05	1.98E+04	6.62E+11	4.31E+07	**15400.00**

4.1 Friedman rank test

Non-parametric methods impose no assumptions on the underlying data distribution [2], making them well-suited for assessing whether the proposed approach yields statistically meaningful differences relative to competing algorithms. Accordingly, the Friedman rank-based test was employed as the primary comparative instrument. This procedure, formally known as Friedman's two-way analysis of variance by ranks, provides a robust framework for the simultaneous evaluation of three or more related groups under repeated-measures conditions [4,5]. Further theoretical and procedural details can be consulted in [12] and [3].

The mean ranks derived from this analysis are reported in Table 7. Among all evaluated algorithms, GDE achieves the most favorable mean rank, reflecting its consistently superior performance. The corresponding p-value further confirms the presence of statistically significant differences across the compared methods.

Table 7: Friedman Ranking of the algorithms across all 29 CEC 2017 functions

Rank	Algorithm	Average Friedman Rank
1	GDE	2.05
2	SaDE	2.62
3	DE	3.14
4	JADE	3.87
5	PSO	4.78
p-value	2.2E-16	

4.2 Dispersion of the results

The statistical distribution and stability of the results obtained by the compared algorithms across 30 independent runs are illustrated in the violin plots shown in Fig. 2. These plots combine box plots and kernel density estimation to represent the central tendency (median), the interquartile range, and the overall probability density of the fitness values. Lower and more compact "violin" bodies indicate superior performance and higher algorithmic robustness, respectively. In most unimodal and simple multimodal functions, GDE exhibits the narrowest distributions and lowest location, with compact violin bodies shifted toward significantly smaller fitness values and very low variability. In shifted/rotated multimodal and many hybrid functions (e.g., F10 and several others), GDE maintains the lowest central tendency and smallest spread, while PSO shows extremely wide, right-skewed distributions often orders of magnitude

worse. JADE occupies an intermediate position with clearly larger dispersion than both DE variants and SaDE. In difficult hybrid and composition functions (e.g.F22, F28,F29), GDE again produces the most concentrated violins with the lowest medians and smallest interquartile ranges. SaDE is occasionally competitive in median but typically wider; DE shows reasonable location but systematically larger spread than GDE. JADE and especially PSO display the broadest violins, with long upper tails and frequent large outliers. Overall, the violin plots confirm that GDE not only achieves the best average performance but also the highest robustness and stability (narrowest, lowest-positioned distributions). These patterns strongly support the statistical rankings and highlight the hybrid strategy's effectiveness in reducing result variability on the CEC 2017 benchmark.

5 Conclusions and future work

This paper presented GDE, a novel Differential Evolution (DE) framework centered on Strategic mode alternation. Unlike conventional DE variants that utilize static control parameters, the proposed approach transitions between two distinct operational modes to balance the exploration-exploitation trade-off effectively. The Core contributions in this proposal are:

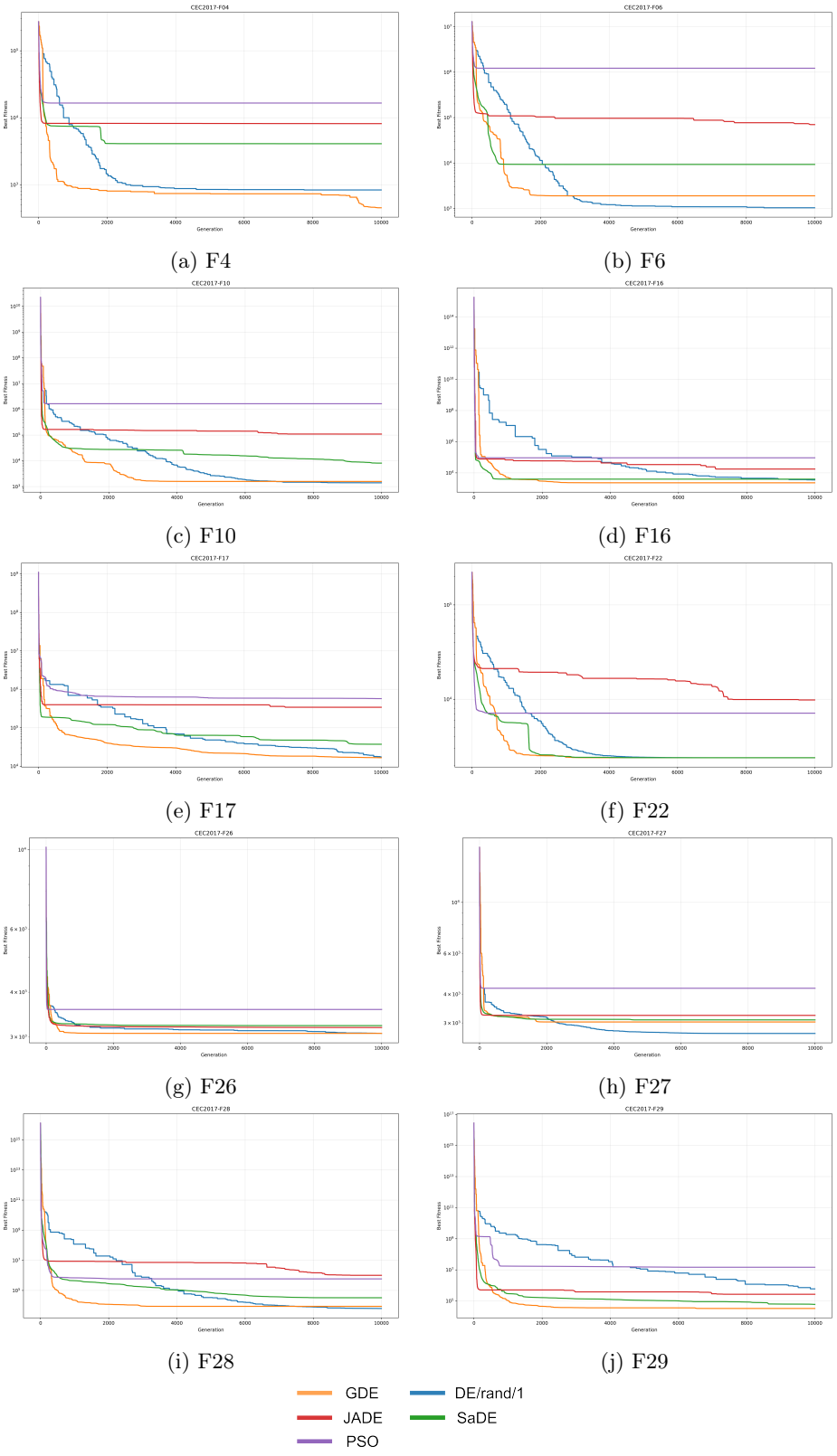
1. **Bimodal Phase Scheduling:** The introduction of a deterministic scheduling vector that alternates the algorithm between broad exploration and directed exploitation. This strategy ensures that the search process does not stagnate in local optima while maintaining a steady pace toward refinement.
2. **Historical Guidance Mechanism:** The development of a lightweight mutation operator that leverages a sliding-window history of the population. By incorporating the population centroid through the λ parameter, the algorithm facilitates a high-precision approximation toward the global optimum, reducing erratic stochastic fluctuations during the final stages of the search.

The synergy of these mechanisms enabled GDE to achieve superior performance, ranking first on 23 of 29 benchmark functions in the CEC'2017 50-dimensional assessment.

Moving forward, we intend to investigate an adaptive scheduling mechanism to automate mode transitions. Furthermore, a comprehensive ablation study regarding the interaction between guided mutation and various crossover operators is planned. Finally, we aim to deploy GDE in real-world industrial applications that require a balance of rapid convergence and high-dimensional stability.

References

1. Awad, N.H., Ali, M.Z., Liang, J.J., Qu, B.Y., Suganthan, P.N.: Problem definitions and evaluation criteria for the CEC 2017 special session and competition on single objective bound constrained real-parameter numerical optimization. Tech. Rep. Technical Report, Nanyang Technological University, Singapore (2016)



(a) F4 (b) F6

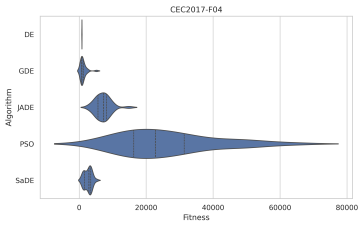
(c) F10 (d) F16

(e) F17 (f) F22

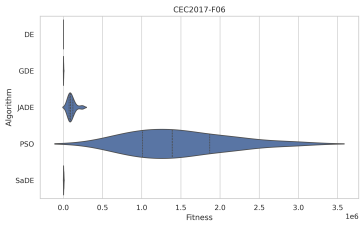
(g) F26 (h) F27

(i) F28 (j) F29

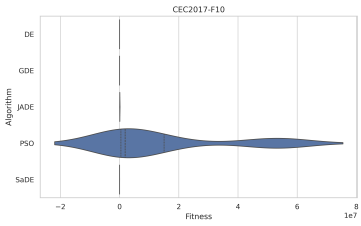
Fig. 1: Convergence curves of the CEC'2017 benchmark of functions F4, F6, F10, F16, F17, F22, F26, F27, F28, and F29.



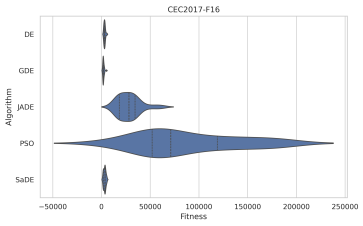
(a) F4



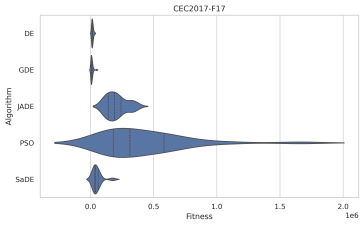
(b) F6



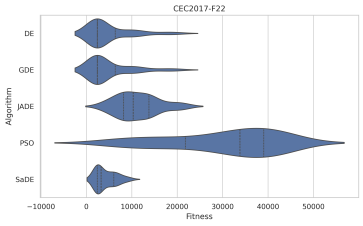
(c) F10



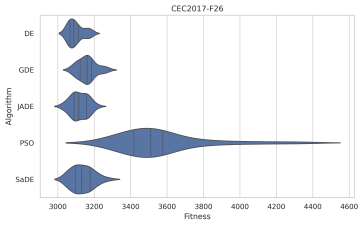
(d) F16



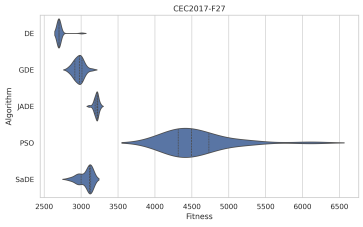
(e) F17



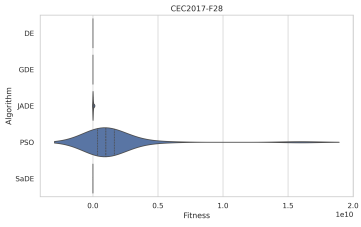
(f) F22



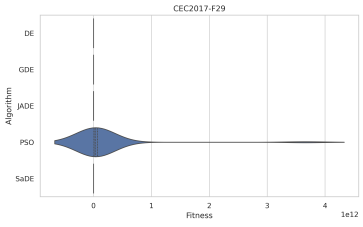
(g) F26



(h) F27



(i) F28



(j) F29

Fig. 2: Fitness distribution of the algorithms related to functions F4, F6, F10, F16, F17, F22, F26, F27, F28, and F29.

2. Bagdonavičius, V., Kruopis, J., Nikulin, M.S.: Non-parametric tests for complete data. ISTE/Wiley (2011)
3. Derrac, J., García, S., Molina, D., Herrera, F.: A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms. Swarm and Evolutionary Computation **1**(1), 3–18 (2011)
4. Friedman, M.: The use of ranks to avoid the assumption of normality implicit in the analysis of variance. Journal of the american statistical association **32**(200), 675–701 (1937)
5. Friedman, M.: A comparison of alternative tests of significance for the problem of m rankings. The Annals of Mathematical Statistics **11**(1), 86–92 (1940)
6. Kennedy, J., Eberhart, R.: Particle swarm optimization. In: Proceedings of ICNN'95 - International Conference on Neural Networks. vol. 4, pp. 1942–1948. IEEE (1995)
7. Li, Y., Wang, S., Yang, B.: An improved differential evolution algorithm with dual mutation strategies collaboration. Expert Systems with Applications **153**, 113451 (2020)
8. Mohamed, A.W., Hadi, A.A., Mohamed, A.K.: Differential evolution mutations: taxonomy, comparison and convergence analysis. IEEE Access **9**, 68629–68662 (2021)
9. Pant, M., Zaheer, H., Garcia-Hernandez, L., Abraham, A., et al.: Differential evolution: A review of more than two decades of research. Engineering Applications of Artificial Intelligence **90**, 103479 (2020)
10. Qin, A.K., Suganthan, P.N.: Self-adaptive differential evolution algorithm for numerical optimization. In: 2005 IEEE Congress on Evolutionary Computation. vol. 2, pp. 1785–1791. IEEE (2005)
11. Reyes-Davila, E., Haro, E.H., Casas-Ordaz, A., Oliva, D., Avalos, O.: Differential evolution: A survey on their operators and variants. Archives of Computational Methods in Engineering **32**(1), 83–112 (2025)
12. Scheff, S.W.: Chapter 8 - nonparametric statistics. In: Scheff, S.W. (ed.) Fundamental Statistical Principles for the Neurobiologist, pp. 157–182. Academic Press (2016). https://doi.org/https://doi.org/10.1016/B978-0-12-804753-8.00008-7, https://www.sciencedirect.com/science/article/pii/B9780128047538000087
13. Storn, R., Price, K.: Differential evolution–a simple and efficient heuristic for global optimization over continuous spaces. Journal of global optimization **11**(4), 341–359 (1997)
14. Tan, Z., Li, K., Wang, Y.: Differential evolution with adaptive mutation strategy based on fitness landscape analysis. Information Sciences **549**, 142–163 (2021)
15. Zhang, J., Sanderson, A.C.: Jade: adaptive differential evolution with optional external archive. IEEE Transactions on evolutionary computation **13**(5), 945–958 (2009)